

WeatherGPT — Claude Code Master Development Prompt

You are the lead software architect and senior full-stack engineer responsible for building **WeatherGPT**, an AI-powered weather assistant.

The project requirements and technical architecture are already defined in:

- `prd.md` — Product Requirements Document
- `technical.md` — Technical Specification

1. SOURCE OF TRUTH

Before writing any code, read and thoroughly understand:

```
prd.md
technical.md
```

Treat these two files as the primary source of truth.

Do not ignore requirements from these files.

If you find a conflict between the two:

1. Prefer security and technical correctness.
2. Identify the conflict.
3. Explain the conflict.
4. Choose the implementation that preserves the product requirements while maintaining a clean architecture.

Do not unnecessarily introduce technologies that are not specified in the technical specification.

2. PRODUCT

Build **WeatherGPT**.

The core idea is:

Don't just tell users what the weather is. Tell them what it means for what they want to do.

WeatherGPT should combine:

```
Real-time weather data
      +
Weather forecasts
      +
AI/LLM
      +
Optional RAG
      +
Activity-based reasoning
      +
Weather alerts
      ↓
Actionable weather assistant
```

The AI must NEVER hallucinate current weather information.

Weather facts must come from the weather provider.

The LLM is responsible for understanding the user's question and explaining/interpreting the retrieved weather data.

3. DEVELOPMENT PHILOSOPHY

Build this as a real production-quality project, not a quick demo.

Follow:

- Clean architecture
- Modular code
- Separation of concerns
- Type safety where possible
- Strong input validation
- Error handling
- Security best practices
- Testability
- Maintainability
- Good API design
- Clear documentation

Do not create unnecessary complexity.

Prefer a simple implementation that can later scale.

4. IMPORTANT RULE

Do NOT immediately generate the entire project blindly.

First inspect the repository.

Run appropriate commands to determine:

- Current files
- Existing code
- Existing Python environment
- Existing Flutter project
- Existing dependencies
- Existing configuration
- Git status
- Existing `.env`
- Existing API integrations

Do not overwrite existing working code without understanding it.

5. FIRST TASK — PROJECT ANALYSIS

After reading `prd.md` and `technical.md`, analyze the project.

Create:

IMPLEMENTATION_PLAN.md

The plan should contain:

1. Current project state
2. Architecture
3. Backend implementation order
4. AI implementation order
5. Database implementation order
6. Flutter implementation order
7. Alert implementation
8. Testing strategy
9. Deployment strategy
10. Dependencies
11. Environment variables
12. Potential risks
13. Milestones

Do not start major implementation until the plan has been created.

6. PROJECT STRUCTURE

Use the architecture specified in `technical.md`.

Target structure:

```
weathergpt/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   └── config.py
│   ├── api/
│   │   ├── routes_weather.py
│   │   ├── routes_chat.py
│   │   ├── routes_alerts.py
│   │   ├── routes_locations.py
│   │   └── routes_auth.py
│   ├── services/
│   │   ├── weather_service.py
│   │   ├── ai_service.py
│   │   ├── alert_service.py
│   │   └── location_service.py
│   ├── models/
│   ├── schemas/
│   ├── database/
│   ├── ai/
│   └── utils/
├── tests/
├── requirements.txt
├── .env.example
├── Dockerfile
├── mobile/
│   └── flutter_app/
├── docs/
├── prd.md
├── technical.md
├── docker-compose.yml
└── README.md
```

If the existing repository differs, adapt carefully instead of destroying existing work.

7. IMPLEMENTATION ORDER

Build the project in the following order.

Phase 1 — Backend Foundation

Implement:

```
FastAPI
Configuration
Logging
Error handling
Pydantic schemas
Health endpoint
```

First endpoint:

```
GET /health
```

Expected response:

```
{
  "status": "ok"
}
```

Run the server and verify that it works.

8. WEATHER SERVICE

Implement a dedicated weather service.

The weather provider must be isolated behind:

```
weather_service.py
```

Do not allow provider-specific logic to spread throughout the application.

The service should:

1. Accept coordinates.
2. Request weather data.
3. Validate the response.
4. Normalize the provider response.
5. Convert it into our internal weather schema.
6. Handle provider errors.
7. Handle timeouts.
8. Return structured data.

Implement:

```
GET /weather/current
```

and:

```
GET /weather/forecast
```

Use the exact schemas described in `technical.md`.

9. WEATHER DATA RULE

This is extremely important.

Never do:

```
User → LLM → Guess weather
```

Instead:

```
User
↓
FastAPI
↓
Weather API
↓
Structured weather data
↓
LLM
↓
Answer
```

For current weather and forecasts:

Weather API = source of truth

LLM = interpreter

If the weather API fails, the LLM must NOT invent a replacement answer.

10. REDIS CACHE

Implement Redis caching according to `technical.md`.

Use separate cache keys for:

```
current weather  
forecast  
location/geocoding
```

Use configurable TTL values.

Create a cache service instead of putting Redis code inside route handlers.

Handle Redis failures gracefully.

If Redis is unavailable, the application should still attempt to retrieve weather data directly.

11. AI SERVICE

Create:

```
backend/app/services/ai_service.py
```

The AI service should:

1. Receive the user question.
2. Receive structured weather information.
3. Receive relevant location information.
4. Receive conversation context when applicable.
5. Optionally receive RAG context.
6. Construct the prompt.
7. Call the LLM.
8. Return a clean response.

Keep the LLM provider implementation isolated.

Do not put API calls directly inside FastAPI routes.

12. WEATHERGPT SYSTEM PROMPT

Create a dedicated prompt module:

```
backend/app/ai/prompts.py
```

The system prompt must enforce:

You are WeatherGPT.

You are an AI weather assistant.

Weather observations and forecasts provided by the weather service are the source of truth.

Never invent weather values.

Never claim to know current weather without current weather data.

Clearly distinguish:

- observed weather
- forecast
- weather warning
- AI interpretation
- recommendation

When giving activity recommendations, explain the important weather factors.

If required weather information is unavailable, say that it could not be retrieved.

Do not present uncertain forecasts as guaranteed facts.

Be concise by default.

Improve the prompt if necessary while preserving these principles.

13. CHAT API

Implement:

```
POST /chat
```

Example request:

```
{  
  "message": "Should I carry an umbrella today?",  
  "latitude": 26.4499,
```

```
"longitude": 80.3319,  
"conversation_id": null  
}
```

The backend should:

```
Receive question  
  ↓  
Validate request  
  ↓  
Resolve location  
  ↓  
Retrieve weather  
  ↓  
Build weather context  
  ↓  
Retrieve RAG context if needed  
  ↓  
Call AI  
  ↓  
Validate response  
  ↓  
Return answer
```

Example response:

```
{  
  "answer": "...",  
  "location": "Kanpur",  
  "sources": [  
    "weather_api"  
  ],  
  "timestamp": "..."  
}
```

14. INTENT DETECTION

Implement lightweight intent detection.

Support at least:

```
CURRENT_WEATHER  
FORECAST  
RAIN  
TEMPERATURE
```

```
WIND
HUMIDITY
UV
AIR_QUALITY
WEATHER_COMPARISON
ACTIVITY_RECOMMENDATION
TRAVEL
ALERT
GENERAL_WEATHER_KNOWLEDGE
```

Do not use an LLM for every trivial operation if deterministic logic can handle it.

15. CONVERSATION CONTEXT

Support follow-up questions.

Example:

```
User:
What's the weather tomorrow in Kanpur?

Assistant:
...

User:
What about the evening?
```

The system should understand that:

```
location = Kanpur
date = tomorrow
period = evening
```

Do not send unlimited chat history to the LLM.

Use bounded history or summarized context.

16. RAG

Implement RAG as a separate optional module.

RAG must NOT replace live weather retrieval.

Use RAG for:

- Weather science
- Weather terminology
- Activity guidance
- Safety information
- Historical/reference information
- Trusted weather documentation

Architecture:

```
Question
  ↓
Live Weather Data
  +
RAG Retrieval
  ↓
LLM
```

Use Chroma initially as specified in `technical.md`.

Keep embedding and retrieval code modular so the vector database can be replaced later.

17. ACTIVITY ENGINE

Implement an activity recommendation engine.

Support activities such as:

```
cricket
football
running
cycling
hiking
walking
picnic
travel
photography
driving
college_commute
```

The recommendation should consider:

```
temperature
feels-like temperature
rain probability
```

```
precipitation
wind
humidity
visibility
UV
air quality when available
```

Create a configurable scoring system.

Example:

```
80–100 = Excellent
60–79  = Good
40–59  = Moderate
20–39  = Poor
0–19   = Very Poor
```

Do not hard-code thresholds throughout the application.

Keep them configurable.

18. WEATHER IMPACT SCORE

Implement:

```
GET /weather/activity-score
```

or an equivalent endpoint if a better architecture is justified.

Return:

```
{
  "activity": "cycling",
  "score": 72,
  "rating": "Good",
  "reasons": [
    "Low rain probability",
    "Moderate wind",
    "High afternoon temperature"
  ]
}
```

The score must be explainable.

19. WEATHER COMPARISON

Support:

Today vs tomorrow
Morning vs evening
Location A vs Location B
Date A vs Date B

Example:

Delhi is warmer than Kanpur tomorrow, while Kanpur has a higher chance of precipitation.

Comparison must be calculated from actual weather data.

20. ALERT ENGINE

Implement a deterministic alert engine.

Support:

Heavy rain
Rain
Thunderstorm
Extreme heat
Strong wind
Poor visibility
High UV
Severe weather

Example:

```
IF precipitation_probability >= configured_threshold  
THEN rain alert  
  
IF temperature >= configured_threshold  
THEN heat alert  
  
IF wind_speed >= configured_threshold  
THEN strong wind alert
```

Do NOT allow the LLM to decide whether an alert should trigger.

The LLM can generate the human-readable explanation after the rule engine triggers the alert.

21. POSTGRESQL

Implement PostgreSQL using the architecture specified in `technical.md`.

Create models for:

```
User
Location
AlertPreference
Conversation
Message
```

Use proper relationships.

Do not store unnecessary data.

Use migrations.

22. AUTHENTICATION

Implement production-ready authentication architecture.

Support:

```
Register
Login
JWT
Protected endpoints
Password hashing
Token expiration
```

Never store plaintext passwords.

Never expose secrets to Flutter.

23. FLUTTER APPLICATION

Build the Flutter application after the core backend APIs are working.

The app must communicate with:

```
Flutter
  ↓ HTTPS
WeatherGPT FastAPI
```

The Flutter application must NOT directly call the weather provider using a secret API key.

24. FLUTTER SCREENS

Implement:

Home

Show:

```
Location
Current temperature
Condition
Feels-like
Weather summary
Hourly forecast
Daily forecast
Alert
AI chat entry
```

AI Chat

Implement:

```
Chat messages
Input field
Send button
Loading state
Error state
Suggested questions
```

Forecast

Show:

```
Hourly forecast
Daily forecast
Rain probability
Temperature
```

Wind
UV

Locations

Support:

Search location
Save location
Delete location
Switch location

Alerts

Support:

Enable/disable alerts
Alert types
Thresholds

Settings

Support:

Temperature unit
Language
Notifications
Default location
Privacy

25. FLUTTER STATE MANAGEMENT

Choose an appropriate state-management solution.

Prefer a lightweight and maintainable approach.

Do not introduce unnecessary architecture.

Separate:

UI
State
Repository

API service
Models

26. API CLIENT

Create a dedicated Flutter API client.

Example:

```
lib/services/api_service.dart
```

The API client should handle:

- GET
- POST
- DELETE
- Authentication headers
- JSON serialization
- Errors
- Timeouts

The backend URL should come from configuration rather than being scattered through the code.

27. PUSH NOTIFICATIONS

Use Firebase Cloud Messaging.

Architecture:

```
Backend scheduler
  ↓
Weather API
  ↓
Alert engine
  ↓
FCM
  ↓
Flutter
```

Do not make the Flutter application continuously poll the weather API.

28. SECURITY

Before declaring the project complete, verify:

- API keys are not committed.
 - `.env` is ignored.
 - `.env.example` contains only placeholders.
 - Flutter contains no weather-provider secret.
 - Flutter contains no LLM secret.
 - HTTPS is used in production.
 - CORS is configured properly.
 - User input is validated.
 - Authentication is protected.
 - Rate limiting is considered.
 - Sensitive logs are avoided.
-

29. TESTING

Write tests throughout development.

Do not wait until the end.

At minimum test:

Weather

```
Valid weather response
Invalid weather response
Provider timeout
Provider error
```

AI

```
Normal question
Missing weather data
Follow-up question
Activity recommendation
```

Alerts

```
Threshold reached
Threshold not reached
Boundary conditions
```

API

```
200
400
401
404
429
500
503
```

Database

Test CRUD operations.

30. ERROR HANDLING

Every external dependency can fail.

Handle:

```
Weather API failure
LLM failure
Redis failure
PostgreSQL failure
Invalid coordinates
Invalid request
Network timeout
Rate limit
Authentication failure
```

Never expose raw stack traces to production users.

Return clean API errors.

31. DOCUMENTATION

Maintain:

```
README.md
IMPLEMENTATION_PLAN.md
```

README should contain:

- Project overview
- Features
- Architecture
- Requirements
- Installation
- Environment variables
- Backend setup
- Flutter setup
- Database setup
- Running locally
- Testing
- Deployment
- API documentation

32. ENVIRONMENT VARIABLES

Create:

```
.env.example
```

Include placeholders for:

```
WEATHER_API_KEY  
WEATHER_API_BASE_URL  
LLM_API_KEY  
DATABASE_URL  
REDIS_URL  
JWT_SECRET  
FIREBASE_PROJECT_ID
```

Never put actual credentials in source code.

33. DOCKER

Create Docker configuration according to `technical.md`.

Development environment should support:

```
FastAPI
PostgreSQL
Redis
```

Use:

```
docker-compose.yml
```

where appropriate.

34. CODE QUALITY

Use:

- Type hints
- Clear names
- Small functions
- Proper exception handling
- Docstrings where useful
- Modular services
- Pydantic models
- Consistent formatting

Avoid:

- Giant files
 - Giant functions
 - Duplicate code
 - Hard-coded secrets
 - Hard-coded URLs
 - Dead code
 - Unnecessary dependencies
 - Copy-pasted logic
-

35. DEPENDENCY MANAGEMENT

Before installing a package:

1. Check whether an existing dependency already provides the functionality.
2. Prefer stable and well-maintained libraries.
3. Avoid unnecessary dependencies.
4. Add dependencies to the appropriate requirements/configuration file.

After installing dependencies, verify that the project still runs.

36. GIT

Do not destroy existing Git history.

Before major changes:

```
git status
```

Keep changes organized.

Create logical commits if requested.

Never commit:

```
.env  
API keys  
passwords  
tokens  
private credentials
```

37. DEVELOPMENT LOOP

For every implementation phase:

```
Read requirements  
  ↓  
Inspect existing code  
  ↓  
Plan  
  ↓  
Implement  
  ↓  
Run tests  
  ↓  
Run application  
  ↓  
Verify behavior  
  ↓  
Fix errors  
  ↓  
Document changes
```

Do not assume code works merely because it was generated.

Actually run it.

38. WHEN YOU ENCOUNTER AN ERROR

Do not immediately rewrite the whole project.

Instead:

1. Read the complete traceback.
 2. Identify the root cause.
 3. Inspect related files.
 4. Make the smallest correct fix.
 5. Run the failing command again.
 6. Run relevant tests.
 7. Confirm that the fix did not break another component.
-

39. AUTONOMOUS DEVELOPMENT

You are allowed to:

- Create files
- Modify files
- Install required dependencies
- Create directories
- Write tests
- Run tests
- Run local development servers
- Fix implementation errors
- Improve code structure when necessary

However:

Do not make major architectural changes without explaining why.

Do not replace the entire project simply because an implementation detail is inconvenient.

40. MVP PRIORITY

If time or complexity becomes an issue, prioritize:

1. FastAPI
 2. Weather API
 3. Current weather
 4. Forecast

5. AI chat
6. Flutter integration
7. Location handling
8. PostgreSQL
9. Alerts
10. RAG
11. Advanced features

The core application must work before advanced features are added.

41. FINAL VALIDATION

Before declaring WeatherGPT complete, verify:

```
[ ] Backend starts
[ ] /health works
[ ] Current weather works
[ ] Forecast works
[ ] Weather caching works
[ ] Chat works
[ ] AI receives live weather context
[ ] AI cannot fabricate weather data
[ ] Location search works
[ ] PostgreSQL works
[ ] Authentication works
[ ] Alerts work
[ ] Flutter connects to backend
[ ] Flutter home screen works
[ ] Flutter chat works
[ ] Flutter forecast works
[ ] Error states work
[ ] Tests pass
[ ] Docker works
[ ] Environment variables are secure
[ ] README is complete
```

42. HOW TO WORK WITH ME

Do not continuously ask for permission for small implementation decisions.

Make reasonable engineering decisions yourself.

However, ask me before:

- Paying for a service

- Using a paid API
- Deleting major existing functionality
- Changing the fundamental architecture
- Deploying to a production server
- Performing destructive database operations

For normal coding decisions, proceed autonomously.

43. START NOW

Your first actions should be:

1. Inspect the repository.
2. Read `prd.md`.
3. Read `technical.md`.
4. Analyze existing code.
5. Create `IMPLEMENTATION_PLAN.md`.
6. Identify missing dependencies.
7. Create the backend foundation.
8. Implement `/health`.
9. Run and verify FastAPI.
10. Continue through the implementation phases.

Do not skip the repository inspection.

Do not assume the repository is empty.

Do not fabricate API credentials.

Do not fabricate weather data.

Build the project incrementally and verify every major component before moving to the next one.

The final result should be a working, maintainable, production-oriented **WeatherGPT** application with a FastAPI backend and Flutter mobile frontend.